Introduction to Comments in C

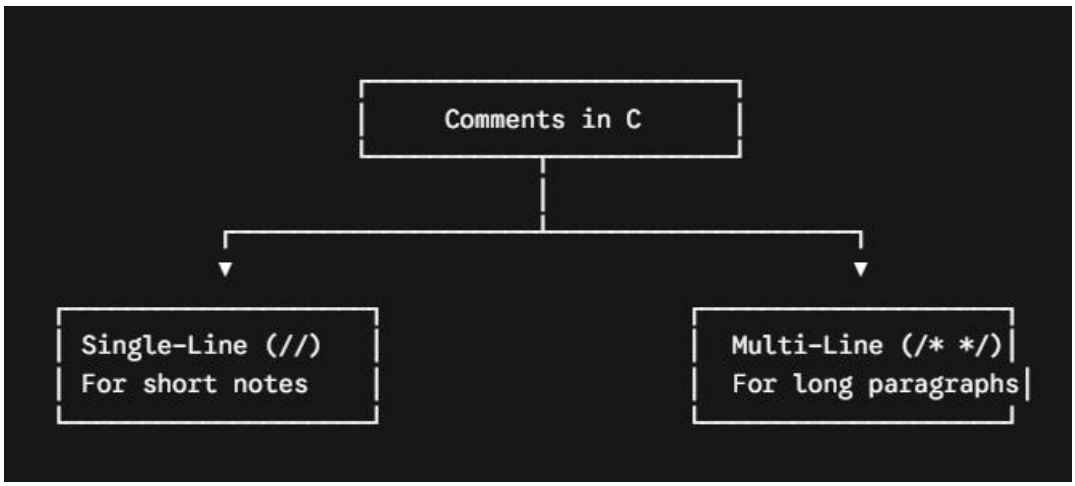
Before diving into control instructions, understanding **comments** is essential. When writing code, you aren't just writing it for the computer—you are also writing it for human beings (your future self, teammates, or instructors).

In C programming, comments are non-executable lines of code used to provide notes, explanations, or documentation inside a source file.

When you compile your program, the compiler completely strips out and ignores all comments. They take up **zero memory** in the final compiled machine code and have absolutely **no impact** on the performance of your program.

Types of Comments in C

C supports two formats for writing comments:



1. Single-Line Comments (//)

Introduced in the C99 standard, single-line comments begin with two forward slashes (//).

Everything following the slashes on that specific line is treated as a comment.

Best used for: Short, quick notes right next to or above a line of code.

Syntax Example:

```
int age = 18;    // Setting the default voting age
// The line below prints a greeting to the terminal
printf("Hello, User!");
```

2. Multi-Line / Block Comments (/* ... */)

Inherited from the original ANSI C (C89) standard, multi-line comments begin with a forward slash and an asterisk (/*) and end with an asterisk and a forward slash (*/).

Anything placed between these two markers can span across multiple lines.

> **Best used for:** Header documentation, explaining complex logic algorithms, or multi-line paragraphs.

Syntax Example:

/* Program Name: Matrix Multiplication Author: Student Developer Date: June 2026 Description: This block calculates the dot product of two arrays. */

🎯 Introduction to Control Instructions

By default, a C program executes sequentially—line by line, from top to bottom. **Control Instructions** are keywords and structures that allow you to break this linear path, enabling you to control, divert, or repeat the execution flow based on specific conditions.

C provides four primary types of control instructions:

- **Decision Control Instructions** (Selection)
- **Iterative Control Instructions** (Loops)
- **Switch-Case Control Instructions**
- **Goto Control Instructions** (Unconditional Jump)

⚡ Truth in C: The Golden Rule

Before writing conditional code, you must understand how C evaluates "Truth":

- **True:** Any **non-zero** value (positive or negative numbers like 1, -5, 100).
- **False:** A value of exactly **zero (0)**.

1. Decision Control Instructions

Used to execute specific blocks of code only when certain logical conditions are met.

A. The if Statement

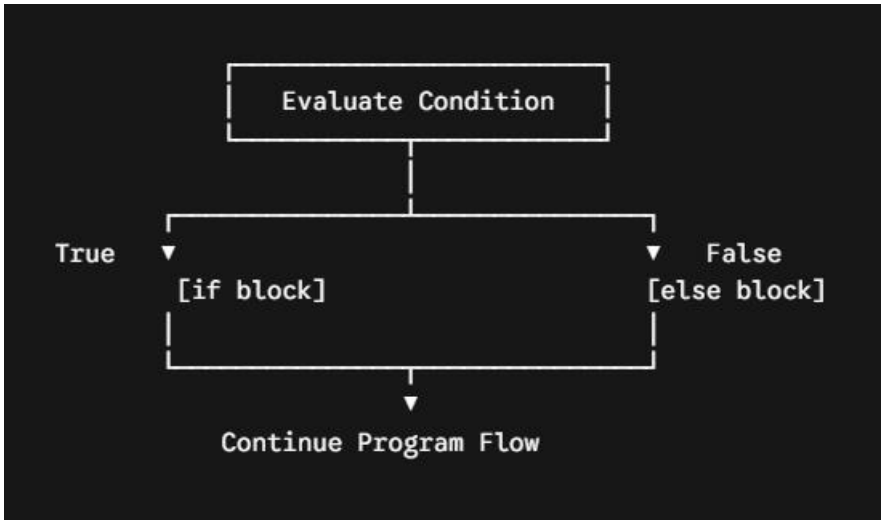
The simplest decision structure. If the condition evaluates to true (non-zero), the inner block runs. If false, it is skipped entirely.

Syntax:

```
if (condition) {
    // Code executes if condition is True (non-zero)
}
```

B. The if-else Statement

Provides an alternate execution path when the if condition fails.



Syntax:

```
if (condition)
{
    // Runs if condition is True
} else
{
```

```
    // Runs if condition is False
}
```

⚠ **The Single-Line Rule:** If there is **only one** statement inside an if or else block, the curly braces { } are optional. However, keeping them is highly recommended to avoid bugs.

💻 Practical Example: Positive or Non-Positive Checker

Here is how you clean up and complete the program from your notes to check if a number is positive or non-positive:

Program :

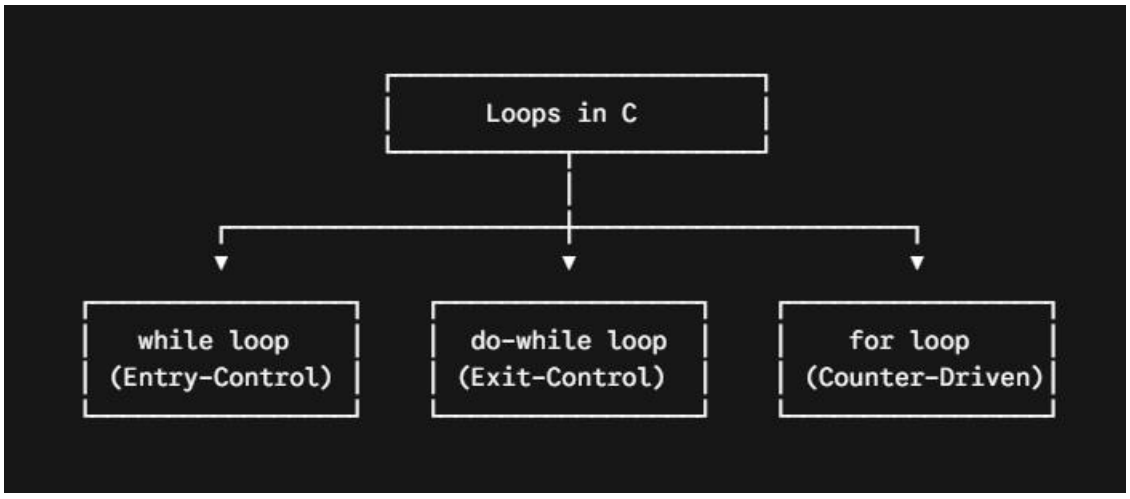
```
#include <stdio.h>   // Standard Input-Output Header
int main()
{
   int x;
   printf("Enter a number: ");
   scanf("%d", &x);                    // Reads input from user

   printf("%s\n", (x > 0) ? "Positive" : "Non-Positive");

   return 0;
}
```

🔁 3. Iterative Control Instructions (Loops)

Loops repeat a block of code as long as a specified condition remains true. C offers three types of loops:



A. while Loop (Entry-Controlled)

The condition is checked **before** entering the loop body. If the condition is false initially, the code block never runs.

Syntax:

```
while (condition)
 {
    // Loop body
}
```

🔍 Critical Code Analysis: The Infinite Loop Danger

```
#include <stdio.h>
int main()
{
   int i = 10;
   while (i)  ----> // 'i' is 10 (non-zero), which means TRUE
    {
         printf("%d ", i);
   }
   return 0;
}
```

Missing: i--;

Behavior Analysis: Because there is no update statement (like i--) inside the loop body, i remains 10 forever. The condition stays **True** infinitely, creating an **infinite loop** that prints 10 10 10... until the program is forcefully terminated.

> **Fix:** Add i--; inside the loop to decrement its value until it hits 0 (False), allowing the loop to stop safely.

B. do-while Loop (Exit-Controlled)

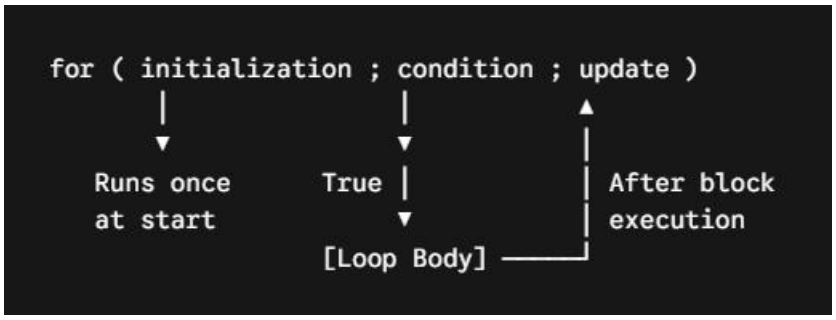
The loop body executes **at least once** because the condition is checked at the very end of the cycle.

Syntax:

```
do
{
    // Loop body (runs at least once)
} while (condition);   // ⚠ Note the mandatory semicolon here!
```

C. for Loop (Counter-Driven)

An optimized loop structure that brings initialization, condition testing, and variable updates together into a single line.



Syntax:

```
for (initialization; condition; update)
{
    // Loop body
}
```

🛑 4. Loop Interrupts: The break Keyword

The break statement is a keyword used to exit a loop or a switch block immediately, skipping any re maining iterations.

Example:

```
int age = 1;
while (age <= 100)
{
   if (age == 18)
   {
```

```
        break;    // Forces immediate exit from the loop when age hits 18
    }
    printf("%d\n", age);
    age++;
}                 // Control jumps straight here after break
```

5. The switch-case Statement

The switch statement acts as a multi-way branch selector, comparing an expression's value against multiple predefined cases.

```
switch (expression)    // Must evaluate to an integer or character
{
    case constant1:     // No variables allowed in case labels
        // Code
        break;              // Prevents fall-through to the next case
    case constant2:
        // Code
        break;
    default:        // Executes if no cases match
        // Code
}
```

Strict Legal Rules for switch in C

Data Type Restriction: The switch expression and case constants can **only** be integers or characters. Floating-point/real constants are strictly **forbidden**.

No Variables in Cases: Case markers must be raw literals or constant definitions (e.g., case 5: or case 'A'). You cannot write case x:

Unique Cases Only: Every single case constant within a single switch block must be entirely unique. Duplicate labels will trigger a compilation error.

The default Block: The optional default block executes if none of the explicit case constants match the input expression. It acts as the catch-all "else" group for switches.

📖 **Stuck or Prefer a Visual Walkthrough?** If you are not able to understand through these notes or you need to explore these concepts more visually, you can review my handwritten notes PDF provided in the same folder : **07 Control instruction**.